Report 1: Q67
Maximum Length of Pair Chain Using Greedy Algorithm

Aim

To develop a program that determines the maximum length of a chain that can be formed from a given set of pairs using the Greedy Algorithm. The objective is to select pairs in such a way that the chain length is maximized while satisfying the chaining condition.

Objective

The purpose of this experiment is to understand how the Greedy technique can be applied to optimization problems. The problem involves selecting pairs such that for every pair (a, b), the next pair (c, d) satisfies the condition $b < c$. The goal is to maximize the total number of pairs included in the chain.

Problem Description

A pair chain is formed when the ending value of one pair is smaller than the starting value of the next pair. Given multiple pairs, not all pairs can necessarily be included in the chain. Therefore, an efficient strategy is required to select the optimal pairs that produce the longest chain possible.

For example, given the pairs:

(1,2), (3,4), (2,3)

The chain (1,2) → (3,4) satisfies the required condition because $2 < 3$. Hence, the maximum chain length is 2.

Algorithm

1. Read the number of pairs.
2. Store all pairs in a list.
3. Sort the pairs according to their second element.
4. Select the first pair and initialize the chain length as 1.
5. Traverse the remaining pairs.
6. If the first element of the current pair is greater than the ending value of the previously selected pair, include it in the chain.
7. Increase the chain count.
8. Continue until all pairs are processed.
9. Display the maximum chain length.

Python Program

```
def max_chain_length(pairs):
    pairs.sort(key=lambda x: x[1])

    count = 1
    last_end = pairs[0][1]

    for i in range(1, len(pairs)):
        if pairs[i][0] > last_end:
            count += 1
            last_end = pairs[i][1]

    return count

n = int(input("Enter number of pairs: "))
pairs = []
```

```
for _ in range(n):
    a, b = map(int, input().split())
    pairs.append((a, b))

print("Max Chain =", max_chain_length(pairs))
```

Sample Input

n = 3

(1,2)

(3,4)

(2,3)

Sample Output

Max Chain = 2

Dry Run

Sorted Pairs:

(1,2), (2,3), (3,4)

Initially select (1,2)

Current chain length = 1

Check (2,3): 2 is not greater than 2, so reject.

Check (3,4): 3 is greater than 2, so select.

Current chain: (1,2) → (3,4)

Maximum chain length = 2

Analysis

The Greedy strategy works by always selecting the pair with the smallest ending value first. This leaves maximum room for future pairs to be added to the chain. Sorting by the second element ensures that the chain can grow optimally. Since each pair is processed only once after sorting, the algorithm is highly efficient.

Advantages

• Simple and easy to implement.

• Provides optimal solution for this problem.

• Requires very little extra memory.

• Efficient for large datasets.

Applications

• Task scheduling problems.

• Activity selection problems.

• Resource allocation systems.

• Job sequencing and planning.

• Event management systems.

Time Complexity

Sorting of pairs: O(n log n)

Traversal of pairs: O(n)

Overall Time Complexity: O(n log n)

Space Complexity

O(1)

Result

The Maximum Length of Pair Chain problem was successfully implemented using the Greedy Algorithm. The program efficiently determines the longest possible chain by selecting pairs based on their ending values. The obtained output for the given sample input is 2, confirming the correctness of the approach.